The Infinite Registry and Indexed-Residue Proof Engine

A Self-Sharpening Architecture for Proof-Route Descent, Positional Recoverability, and Certified Registry Enlargement - Version 2

Lu Semita

EmergenceByDesign

July 2026 - Version 2

LABORATORY NOTES RECORD

Independent progress record; open to correction and extension.
Not a claim of ownership, priority, or completed universal theoremhood.

Documentation and Scope Disclosure

This laboratory note records an independent line of inquiry developed within the Lambda / NSAF / TUFT research program. It does not claim ownership, priority, or discovery over automated theorem proving, proof assistants, proof enumeration, proof complexity, Hilbert systems, natural deduction, ZFC, Nock, term rewriting, shortest-path algorithms, graph search, inverse limits, formal verification, or any established mathematical or computational subject referenced here.

The original contribution proposed in this note is architectural and methodological. It combines: (i) an explicitly indexed residue calculus; (ii) an infinite but computably presented registry of finite proof objects and proof-search histories; (iii) deterministic proof verification; (iv) granular proof-cost evaluation; (v) graph-theoretic shortest-route analysis over lemma combinations; and (vi) a self-sharpening loop in which information about unsuccessful, redundant, or expensive proof routes informs later representation-class enlargement.

The paper distinguishes three levels of claim throughout:

Tier I - Standard and checkable: established facts of formal logic, recursive enumerability, proof verification, graph search, conservative derived rules, and the limitations of sufficiently expressive recursively axiomatized theories.

Tier II - Structural interpretation: exact correspondences proposed after Tier-I objects have been fixed, including the interpretation of search failure data, proof compression, and class-relative minimality as indexed residue phenomena.

Tier III - Research program: the proposed Infinite Registry and Indexed-Residue Proof Engine, its self-sharpening architecture, its Nock granularization, and the planned browser demonstrator. These are offered as a precise construction program, not as completed results.

Abstract

This note proposes a formally disciplined proof-engine architecture in which the outcome of proof search is not reduced to a pass/fail bit. Instead, every search stage contributes structured information about what failed, what repeated, what compressed, what remained expensive, and which representation-class enlargement lowered the indexed obstruction. The system begins with a fixed recursively axiomatized theory, ultimately ZFC, and a minimal trusted kernel capable of verifying finite proof objects. Candidate derivations are generated into a computably presented infinite registry, organized as compatible finite stages. Each target theorem is evaluated relative to an explicitly declared proof representation class R, a cost or obstruction functional O, and an indexed residue I_R defined as the infimum of O over all admissible proof representations in R.

The central engineering claim is modest but substantial: a proof environment can become self-sharpening without becoming self-authorizing. It may discover recurring subproofs, propose lemmas and derived rules, reorganize search, and shorten certified proof routes, while every accepted enlargement remains reducible to the immutable primitive calculus. The theorem set therefore remains unchanged under conservative representation-class enlargement even as proof cost decreases. Failure information becomes data for the next iteration rather than disappearing as an undifferentiated rejection.

The broad theory runs from an infinite registry of finite proof objects toward a universal ZFC proof enumerator, verifier, and compression environment. The demonstrator deliberately runs in the opposite direction: it uses a minimal propositional calculus, one target formula, two proof classes, one cost functional, and a certified strict decrease in shortest proof length. A JavaScript and Three.js interface can visualize the proof registry as a graph, animate the search frontier, display rejected and redundant routes, and show how a certified lemma changes the shortest path without changing theoremhood. This smallest implementation suffices to demonstrate the core claim in a reproducible and falsifiable form.

1. Governing Thesis

The governing thesis is:

A formal proof-search system becomes materially more informative when it records the structure of failure, redundancy, and cost relative to a declared representation class, rather than returning only success or failure. Because this information can guide conservative enlargement of the proof representation class, the system can sharpen itself while preserving a fixed trusted kernel.

The architecture distinguishes three questions that ordinary pass/fail interfaces often conflate:

1. Theoremhood: does a valid proof exist in the underlying theory?
2. Representation cost: how expensive is the least proof in the declared proof language?
3. Search state: what has the current finite process established or failed to establish so far?

These quantities are not interchangeable. A theorem may be provable while its shortest proof remains unknown. A proof may become dramatically shorter after adding a certified macro even though the theorem set does not change. A finite search may fail to find a proof without licensing a nonprovability claim. The proposed engine exposes these distinctions directly.

The idea parallels the previously developed grading-descent method: the system does not discard unsuccessful outputs. It grades them. Every rejected proof object carries a reason; every expensive path carries a cost; every recurring fragment carries compression potential; every unresolved target carries a precisely bounded search history. The failure state therefore supplies information about the next admissible transformation.

2. Prior Framework Basis

The present proposal draws on four previously recorded components of the Lambda / NSAF / TUFT canon.

First, The Residue Thesis defines an obstruction O, an indexed residue I_R obtained by minimization over a declared representation class R, and an apparent component A = O - I_R. It emphasizes that class enlargement can only lower or preserve the indexed residue and that pointwise failure alone does not prove positive residue.

Second, Operator-Agnostic Geometry, Topological Obstruction, and Why Completeness Is Not Comprehension states the same orbit-minimized obstruction calculus and separates standard obstruction results from framework interpretation. That work supplies the discipline required here: every positive residue claim must identify either quantitative separation or attained minimization.

Third, Recursive Registry Completion distinguishes a completed finite-dimensional state from the larger registry of compatible histories. It formalizes an infinite registry as a constrained inverse system rather than an unstructured infinite accumulation. The present proof engine adopts the same organizational principle: each finite stage remains available, bonding maps preserve stage compatibility, and the full registry denotes the computable family of all finite proof objects and proof-search histories.

Fourth, Structural Registry of Nested Geometries places incidence before measurement and interpretation. Applied here, proof incidence comes before proof cost. Premise-conclusion relations, substitutions, rule applications, dependency edges, and lemma reuse define the structural registry. Only after that registry exists do we assign step counts, formula-occurrence counts, depth, expansion size, or other cost measures.

The present work therefore does not import the word registry as metaphor. It gives the proof environment an explicit incidence structure, a metric layer, a ratio and comparison layer, and a projection from full proof histories into summarized observations.

3. Formal Setting

Let S be a fixed recursively axiomatized formal theory. The eventual target is ZFC, but the minimal demonstrator uses a finite propositional calculus.

Definition 3.1 (Formula language). Let Form(S) denote the set of well-formed formulas of S under a fixed canonical encoding. In the propositional demonstrator, formulas are generated from atomic symbols and implication. In the ZFC implementation, formulas include equality, membership, logical connectives, and quantifiers.

Definition 3.2 (Primitive proof object). A primitive proof object p is a finite rooted directed acyclic graph, or equivalently a finite tree with shared subterms permitted, whose leaves are axiom instances or declared premises and whose internal nodes are applications of primitive inference rules. Its root is the conclusion conc(p).

Definition 3.3 (Trusted kernel). The kernel K is a deterministic program that decides whether a finite candidate p is a valid primitive proof object for S. K does not search for proofs, rank heuristics, invent rules, or accept statistical confidence. It only checks syntax, schema instantiation, substitution hygiene, and primitive inference validity.

Definition 3.4 (Representation class). For a target formula phi, a proof representation class R(phi) is a declared set of admissible finite proof descriptions whose primitive expansions, when accepted, verify phi in S. A class may contain only primitive proofs or may additionally contain certified macros, named lemmas, compressed subproofs, alternate normal forms, or restricted search grammars.

Definition 3.5 (Obstruction functional). An obstruction or cost functional is a computable map

$$O(p, phi; R, c) >= 0,$$

where c names the selected metric. Typical choices include number of primitive inferences, compressed inference count, number of formula occurrences, proof depth, symbol count, dependency count, maximum lemma rank, or a weighted combination.

Definition 3.6 (Indexed proof residue). For a target phi and representation class R,

$$I_{R,c}(\phi) = \inf \{ O(p, \phi; R, c) : p \in R(\phi), K(\text{expand}(p)) = \text{accept} \}.$$

When no proof exists in R(phi), the infimum is taken as infinity. When R(phi) is finite or when a bounded finite sublevel set has been exhaustively enumerated, an attained minimum can be certified.

Definition 3.7 (Stage-indexed best value). If E_N is the finite enumeration stage N, define

$$b_N(\phi; R, c) = \min \{ O(p) : p \text{ appears by stage } N \text{ and } K(\text{expand}(p)) = \text{accept} \}.$$

If no proof has appeared, b_N = infinity. The sequence b_N is nonincreasing, but b_N = infinity at a finite stage does not imply I_R,c(phi) = infinity.

4. The Infinite Proof Registry

The phrase infinite registry does not mean that an actually infinite list is stored in memory. It denotes a recursively enumerable universe together with its finite stages and compatibility maps.

Let P_N be the finite set of candidate proof objects generated through stage N under a canonical enumeration. Let $i_{N,N+1} : P_N \to P_{N+1}$ be the inclusion preserving every previously generated object. Then

$$P_0 \to P_1 \to P_2 \to \dots$$

forms a directed system. Its union contains every finite candidate proof object that the enumeration is designed to reach. If the grammar enumerates every finite proof term of S, every valid finite proof eventually occurs.

The registry stores more than accepted proofs. For each candidate p it records:

- canonical identifier and hash;
- target or conclusion;
- rule incidence and dependency edges;
- kernel verdict;
- rejection reason, when invalid;
- cost vector;
- duplicate or equivalence-class identifier;
- primitive expansion, if compressed;
- stage of discovery;
- search strategy that generated it;
- subproof-frequency statistics;
- relation to the current best witness.

This is the decisive departure from binary proof search. An invalid object is not retained as mathematics, but its failure mode remains valid data about the search process. A redundant valid proof is not the current optimum, but it remains data about alternative routes, recurring fragments, and lemma candidates.

The complete proof registry may be viewed at several projections:

4. Object projection: all finite proof objects.
5. Theorem projection: the set of conclusions currently certified.
6. Minimum-cost projection: the best known proof per conclusion.
7. Search-history projection: the routes by which candidate objects arose.
8. Residual projection: the distinctions lost when only a pass/fail or best-cost summary is retained.

The full registry therefore contains both proof content and process content.

5. Granularization and Polynomial Grain Sizing

The engine requires a declared computational grain. Without one, claims about shortest paths or complexity silently compare unlike representations.

A grain is the smallest operation counted as one unit under the selected cost model. Examples include:

- one primitive inference;
- one kernel reduction step;
- one Nock reduction;
- one formula occurrence;
- one edge traversal in a proof dependency graph;
- one symbol processed;
- one bounded substitution operation.

The minimal demonstrator should use primitive inference count because it is transparent and auditable. Later implementations may use a cost vector

$$c(p) = (s(p), d(p), f(p), e(p), t(p)),$$

where s is inference count, d is depth, f is formula occurrences, e is primitive expansion size, and t is measured execution time under a fixed evaluator.

Polynomial grain sizing means that every accepted macro or lemma must expose a primitive expansion whose verification cost is polynomially related to its explicit encoded size under the declared machine model. This does not settle P versus NP and should not be presented as doing so. It supplies a disciplined normalization condition: compressed proof notation cannot hide an unbounded or undefined verification burden.

For the browser demonstrator, the comparison metric should remain fixed across classes. If R_1 introduces a macro, two values should be displayed:

- compressed cost in R_1;
- primitive expansion cost in the common kernel language.

This prevents a false improvement created only by changing the ruler. A genuine representation gain appears when the macro lowers search and description cost while preserving a checkable primitive expansion.

6. Proof Space as a Structured Relation Graph

A proof registry naturally induces a directed hypergraph.

Vertices represent formulas, lemmas, proof states, or proof objects. Hyperedges represent rule applications from one or more premises to a conclusion. A proof is a rooted acyclic sub-hypergraph terminating at the target formula.

Let G_R(phi) be the proof hypergraph generated by representation class R for target phi. Assign each hyperedge e a nonnegative cost w(e). Then a shortest proof problem becomes a minimum-cost derivation problem over G_R(phi).

Derived lemmas alter the graph without necessarily altering theoremhood. A certified lemma inserts a shortcut edge whose primitive expansion is already represented by a longer path. Thus the graph changes from G_R0 to G_R1 while the reachability relation from axioms to theorems remains extensionally unchanged.

This makes conservative class enlargement visually exact:

- before enlargement, the target is reached through a long primitive path;
- after enlargement, a certified shortcut contracts a recurrent subgraph;
- the theorem was already reachable;
- the shortest represented route becomes shorter;
- the primitive kernel can expand the shortcut back into the original route.

Lemma combination can then be evaluated as a shortest-path problem. Candidate lemma sets L are scored by the reduction they induce across a target family Phi:

$$\text{Gain}(L) = \sum_{\phi \in \Phi} \left[b_R(\phi) - b_{R+L}(\phi) \right],$$

subject to proof-certificate and expansion constraints. The self-sharpening system seeks lemma sets that maximize certified aggregate route reduction without changing the underlying theory.

7. Failure Telemetry as Mathematical Process Data

The architecture treats failure data as information about the proof-search process, not as proof of mathematical impossibility.

Every unsuccessful candidate is classified. A minimal taxonomy includes:

F1 - syntactic failure: the object is not a well-formed proof term.
F2 - schema failure: a claimed axiom instance does not match its schema.
F3 - inference failure: a conclusion does not follow from the declared premises under the named rule.
F4 - substitution or binding failure: variable capture, type mismatch, or illegal substitution occurs.
F5 - dependency failure: a required premise or lemma is unavailable in the declared class.
F6 - duplication: the candidate is equivalent to an existing proof under the selected normal form.
F7 - domination: the candidate proves the target but costs no less than an already known witness.
F8 - frontier exhaustion: every candidate within the declared finite bound has been checked without finding a proof.
F9 - unresolved nontermination: the unbounded search remains open.

Only F8 supports a bounded minimality or bounded nonexistence statement. F9 supports no positive residue claim by itself.

The telemetry informs later stages in several ways:

- repeated F5 patterns suggest missing lemmas;
- repeated long subgraphs suggest compression macros;
- F6 clusters suggest stronger canonicalization;
- F7 families reveal alternate proof geometries;
- high branching with low yield suggests search-priority revision;
- stable lower bounds across exhausted finite sublevels support certified stage-relative statements.

This is the proof-engine analogue of grading descent: the system learns from the shape and location of failure rather than merely recording that failure occurred.

8. Self-Sharpening Without Self-Authorization

The self-sharpening layer may propose changes, but it may not certify itself.

Definition 8.1 (Certified derived rule). A derived rule D is certified relative to S when the kernel accepts a primitive proof schema showing that every application of D expands to a valid primitive derivation in S.

Definition 8.2 (Conservative representation enlargement). R_1 is a conservative representation enlargement of R_0 when every R_1 proof expands effectively into an R_0 proof of the same conclusion.

Proposition 8.1 (Theorem preservation). If R_1 conservatively enlarges R_0, then the set of theorems represented by R_1 equals the set represented by R_0.

Proof. Every R_0 proof is admitted in R_1 because R_0 is contained in R_1. Every R_1 proof expands into a valid R_0 proof by conservativity. Therefore each class represents exactly the same theorem set.

Proposition 8.2 (Indexed monotonicity). Under a fixed common cost functional applied to admissible represented routes,

$$I_{R1,c}(\varphi) \leq I_{R0,c}(\varphi).$$

Proof. The minimization domain for R_1 contains that of R_0.

A strict decrease therefore certifies improved representation or search economy, not new deductive power.

The sharpening loop is:

9. enumerate and verify;
10. collect proof and failure telemetry;
11. mine recurring subproofs and high-cost bottlenecks;
12. propose lemmas, macros, normal forms, or search priorities;
13. certify every logical shortcut by primitive expansion;
14. rerun the registry under the enlarged class;

15. compare indexed minima and proof-route structure;
16. retain only improvements that remain reproducible under the fixed kernel.

The kernel remains immutable across this cycle. The periphery adapts; the verifier does not.

9. ZFC as the Broad Registry Domain

The expansive theory targets the universe of machine-decidable finite ZFC proof objects.

This phrase requires precision. For every finite candidate proof string, the kernel can decide whether it is a valid ZFC proof. The set of valid finite ZFC proofs is recursively enumerable. Therefore a universal enumeration can eventually list every finite ZFC proof under a canonical syntax.

The following are achievable:

- enumerate all finite well-formed ZFC proof candidates;
- verify each finite candidate deterministically;
- index valid proofs by theorem, cost, dependency graph, and representation class;
- identify shortest proofs within exhausted finite cost bounds;
- discover conservative lemmas and macros;
- maintain an indefinitely extensible registry of search stages;
- export machine-checkable certificates.

The following are not asserted:

- a terminating decision procedure for every ZFC sentence;
- a complete decision procedure for independence;
- an unrestricted proof of ZFC's own soundness from within the same fixed theory;
- a solution to P versus NP.

These limitations do not reduce the system to a conventional prover. The proposed contribution lies in making representation-relative cost, failure structure, registry history, and conservative self-sharpening first-class formal objects.

10. Nock Granularization

Nock supplies a minimal universal combinator language in which code and data share the same noun representation. This makes it suitable for a later, maximally explicit realization of the registry.

A proof noun can encode:

[proof theory-version target rule premises metadata]

A formula noun can encode atomic formulas, connectives, quantifiers, and variables as binary trees of atoms. A rule noun can encode the primitive checker invoked and the positions of premises within the subject.

The Nock layer serves four purposes:

17. Canonical granularity. Every formula, rule, proof, and search state becomes a noun.
18. Reproducible reduction. Kernel evaluation reduces through a small universal semantics.

19. Content addressability. Proof nouns can receive stable hashes and be deduplicated.
20. Reflective registry. Search procedures, rule libraries, and proof objects inhabit one uniform representational substrate.

Nock does not provide soundness by itself. Soundness remains a property of the encoded kernel and axioms. Nor does Nock bypass incompleteness or computational complexity. Its role is representational austerity: it minimizes hidden machinery and makes the granularization claim tangible.

The recommended implementation sequence remains:

- ordinary JavaScript or Python proof engine;
- canonical noun encoding of all data;
- Nock-compatible primitive evaluator;
- optional Hoon or pure Nock implementation;
- equivalence tests between the ordinary and noun-level kernels.

11. Minimal Demonstrator

The demonstrator should intentionally instantiate only the minimum structure needed to establish the core claim.

Target formula:

$$\text{phi} = (P \to Q) \to ((Q \to R) \to (P \to R)).$$

Representation class R_0:

- a fixed finite Hilbert axiom basis;
- modus ponens;
- no derived rules;
- no lemma library.

Representation class R_1:

- every component of R_0;
- one certified hypothetical-syllogism macro, or an equivalent certified lemma.

Obstruction functional:

$$O(p) = \text{number of represented inference nodes},$$

with a second displayed value equal to primitive expansion size.

Required outputs:

- shortest certified R_0 proof found within an exhaustively completed bound;
- shortest certified R_1 proof within the corresponding bound;
- primitive expansion of the R_1 macro proof;
- proof that R_1 is conservative over R_0 for this rule;
- strict or non-strict comparison I_R1 <= I_R0;
- complete search counts and rejection taxonomy;
- a machine-readable certificate bundle.

The demonstrator must not display "unprovable" when it only means "not found through stage N." It should display:

No proof found through the exhausted bound B. No unrestricted nonprovability claim is licensed.

This visible refusal forms part of the demonstration.

12. JavaScript and Three.js Interface

A single-page application can implement the first demonstrator without a server.

The left panel declares the experiment:

- target formula;
- proof class;
- search bound;
- cost metric;
- optional lemma set.

The central Three.js scene displays the proof registry as a layered directed graph:

- axioms at the base;
- derived formulas at successive inference depths;
- valid edges as illuminated relations;
- rejected candidates as dimmed or collapsed branches;
- the current best proof as a highlighted route;
- repeated subproofs as clustered motifs;
- a proposed lemma as a shortcut arc;
- R_0 and R_1 as switchable graph layers.

The right panel displays residue and telemetry:

- best cost by stage;
- accepted and rejected candidate counts;
- rejection categories;
- proof witness;
- primitive expansion;
- class-enlargement type;
- certificate hash;
- status statement with its exact scope.

An animation should first show the primitive search, then introduce the certified lemma, contract the recurrent subgraph, and recompute the shortest route. The viewer sees that the target did not change, the primitive proof did not become false, and theoremhood did not expand. What changed was the representation class and the shortest represented route.

13. Core Theorem Package for the Paper

The first implementation can support a compact theorem package.

Theorem 13.1 (Finite bounded attainment). Let R_B(phi) be the finite set of admissible proof objects of cost at most B under a finite proof alphabet and decidable verifier. If at least one

member proves phi, then the minimum proof cost in R_B(phi) is attained and exhaustive enumeration certifies it.

Theorem 13.2 (Conservative theorem preservation). A representation-class enlargement in which every new rule has a kernel-verified primitive expansion preserves the theorem set.

Theorem 13.3 (Monotonicity under class enlargement). If R_0 is contained in R_1 and the same cost semantics applies, then I_R1,c(phi) is no greater than I_R0,c(phi).

Theorem 13.4 (Certified strict descent). If the engine exhibits an R_1 proof p_1 with cost less than the certified minimum in an exhaustively enumerated R_0 sublevel, then the indexed proof residue strictly decreases over that declared comparison domain.

Theorem 13.5 (Stage honesty). Failure to find a proof in a finite stage licenses only the statement that no proof occurred in that stage or exhausted sublevel. It does not establish nonprovability in the unbounded class.

Theorem 13.6 (Self-sharpening soundness). If the adaptive layer can alter only search ordering, canonicalization, or certified conservative macros, while acceptance remains determined by the immutable primitive kernel, then adaptive sharpening cannot introduce an invalid theorem into the accepted registry unless the kernel or its primitive axioms are defective.

These results are elementary by design. Their value lies in their assembly into one executable system and one visible residue calculus.

14. Positional Recoverability as a Second Residue Instrument

The proof-route demonstrator measures how a conservative enlargement of a proof language lowers the shortest represented route to a fixed theorem. Positional Recoverability in Binary-Addressed Registries supplies a complementary instrument. Instead of asking how cheaply a proof can be expressed, it asks how much of a relation among registry entries can be recovered from their addresses alone.

Let V be a finite set of registry entries, A the Nock axis set, and lambda: V -> A an injective layout. A declared positional vocabulary Pi consists of predicates pi_i: A x A -> D_i. A relation F: V x V -> W is Pi-recoverable when a decoder delta reconstructs F(u,v) from the tuple of address predicates applied to lambda(u) and lambda(v).

The irreducible positional supplement is the minimum number of unordered pairs whose relation values must be carried outside the positional schema:

res_Pi(F) = min over layouts, decoders, and supplements of the number of supplemented unordered pairs.

This quantity is indexed to the declared vocabulary. It does not measure a defect in the structure or in the registry. It measures the mismatch between the relation and the positional primitives admitted by Pi.

The central symmetry obstruction is exact. If every predicate in Pi is symmetric in its two address arguments, then every Pi-recoverable relation must also be symmetric. Therefore any antisymmetric pair in F contributes at least one unavoidable supplementary datum. This converts

failed layout search into a proof that no layout and no decoder can remove the mismatch while the vocabulary remains symmetric.

15. Exact Calibration on Finite-Type Cartan Data

The positional instrument is calibrated on the Cartan matrices of the finite-dimensional complex simple Lie algebras of rank at most eight. Their support graphs are finite trees of maximum degree three, so each Dynkin diagram can be embedded into the binary Nock-axis tree with diagram adjacency equal to parent-child adjacency.

With the positional vocabulary Pi_0 containing tree adjacency alone, the decoder recovers diagonal entries, zero entries, and all simply-laced off-diagonal entries directly from position. For every simply-laced finite type, the positional residue is zero. Every non-simply-laced finite type has exactly one unordered pair whose two directed Cartan entries differ, and the symmetry obstruction forces at least one supplementary datum. Supplying that one pair reconstructs the matrix exactly. Hence the lower bound is attained:

$$\mathrm{res}_{\Pi_0}(C) = |\mathrm{Asym}(C)|,$$

which equals zero for the simply-laced types and one for the non-simply-laced types in the calibration family.

This result strengthens the proof-engine proposal in a different direction. The proof-route experiment demonstrates that a certified representational enlargement can lower shortest proof cost. The positional experiment demonstrates that a declared representational vocabulary can possess a proven lower bound, identify the exact relations responsible for the shortfall, and reach closure when the missing datum or primitive is supplied.

16. The Combined Self-Sharpening Loop

The two instruments together provide more than two parallel examples. They define a dual self-sharpening cycle.

Proof-route descent asks:

Which certified lemma combinations shorten the path from premises to target?

Positional recoverability asks:

Which relations among proof fragments can be reconstructed from registry structure, and which must remain explicit code or supplementary data?

A combined iteration proceeds as follows:

21. Enumerate and verify proof candidates in a fixed primitive calculus.
22. Retain failure telemetry, repeated subproofs, dependency bottlenecks, and the current shortest accepted routes.
23. Propose a conservative lemma or macro whose primitive expansion is kernel-checkable.
24. Recompute proof-route residue and record any certified descent.
25. Place proof fragments and lemma dependencies into the binary-addressed registry.

26. Probe which dependency, direction, and composition relations become positional under the declared vocabulary.
27. Record the exact positional supplement where address structure cannot recover a relation.
28. Enlarge either the proof representation class or the positional vocabulary only through an explicit, costed, and auditable admission.
29. Repeat with the new class while retaining every prior stage and certificate.

The first descent reduces represented proof cost. The second identifies whether the newly useful relations have become structural in the registry or still require explicit coding. A useful lemma can therefore be evaluated twice: by how much it shortens a proof route and by how economically its dependency relations can be recovered from the registry layout.

This yields a two-coordinate sharpening record:

$$J_n(\phi) = (I_{proof,n}(\phi), \mathrm{res}_{\Pi_n}(F_n)).$$

A sharpening step is not required to decrease both coordinates. A lemma may shorten proof routes while increasing registry supplement, or a better layout may reduce positional supplement without changing theorem proof cost. The architecture records the trade rather than hiding it in a single scalar.

17. Dual Monotonicity and Costed Enlargement

Two elementary monotonicity laws govern the combined engine.

Proof-class monotonicity. If R_0 is contained in R_1 and the same obstruction functional is used, then $I_{R1}(\phi) \le I_{R0}(\phi)$.

Vocabulary monotonicity. If Π_0 is contained in Π_1, then $\mathrm{res}_{\Pi 1}(F) \le \mathrm{res}_{\Pi 0}(F)$, because every layout and decoder available under Π_0 remains available under Π_1.

These laws license descent only relative to enlargement. They do not say that enlargement is free. A new lemma, macro, address predicate, or directional primitive has an admission cost. Version 2 therefore distinguishes three quantities:

- route cost: the cost of proving the target in the represented proof language;
- supplement cost: the explicit relation data that address structure fails to recover;
- vocabulary cost: the complexity of the primitives admitted to make relations positional.

A mature optimizer should seek a Pareto frontier rather than an unqualified minimum. It should not erase a positional residue merely by admitting a predicate equivalent to the entire relation, nor erase proof cost by naming the target theorem as a primitive lemma. Every class enlargement must be recorded with its expressive and storage cost.

This discipline turns self-sharpening into measured re-representation rather than unconstrained accumulation.

18. Version 2 Demonstrator Specification

The Version 2 standalone demonstrator contains two independently executable laboratories and one combined interpretation.

Laboratory A - Indexed proof-route descent.

The browser exhaustively enumerates typed implicational proof terms through a declared node bound, verifies primitive expansions, compares R_0 with a conservatively enlarged R_1, and exports the shortest certified witnesses together with failure telemetry.

Laboratory B - Positional recoverability probe.

The browser lays the finite-type Cartan data onto Nock axes, reconstructs matrix entries from adjacency alone, marks the exact supplementary pairs, verifies closure after supplementation, and sweeps the full calibration family. It displays address width, positional edges, asymmetric pairs, and measured residue.

Combined interpretation.

The page explains that Laboratory A proves a strict descent in represented proof cost while Laboratory B proves a tight lower bound and exact supplement in a distinct registry measurement. Their conjunction demonstrates both sides required for a self-sharpening system: improvement through certified enlargement and information-rich resistance when the current representation cannot absorb a relation.

The demonstrator remains deliberately minimal. It does not yet place the generated proof fragments themselves into an optimized Nock-axis layout or learn positional vocabularies automatically. Those operations become the immediate Version 3 experiment. Version 2 establishes the two calibrated instruments that such an adaptive loop requires.

19. Experimental Program

The paper should report three experiments.

Experiment A - Conservative proof compression.

Run the implication-transitivity target under R_0 and R_1. Demonstrate a shorter represented route after adding hypothetical syllogism. Display the primitive expansion to show theorem preservation.

Experiment B - Complete finite separation.

Use a finite model-checking or primality task in which the candidate class is genuinely exhausted. This supplies a clean residue-zero/residue-positive contrast without confusing bounded search with universal nonexistence.

Experiment C - Open-ended registry growth.

Choose a theorem whose proof is discoverable by staged search. Record the best-cost sequence b_N, proof-family diversity, rejection statistics, recurring subproofs, and lemma proposals. Include at least one unresolved target whose status remains explicitly stage-indexed.

The main quantitative plots should include:

- best proof cost versus stage;
- candidate count versus accepted count;
- rejection taxonomy distribution;

- number of unique versus duplicate proof objects;
- route reduction produced by each certified lemma;
- compressed versus primitive-expanded cost;
- registry size and search-frontier width.

20. Relation to Topological-Geometric Grading Descent

The proof engine shares a methodological form with topological-geometric grading descent.

A binary evaluator collapses an attempted object into pass or fail. A graded evaluator preserves where and how the object failed. Those differences become coordinates for the next transformation.

In geometry, the data may concern closure defect, orientation mismatch, incidence failure, or projection loss. In proof search, the data concern schema mismatch, dependency absence, repeated subgraphs, dominated paths, or frontier exhaustion. In both cases, the next iteration is informed by a structured residual object rather than an undifferentiated negative verdict.

The correspondence can be stated without claiming identity:

- geometric state -> proof state;
- admissible transformation -> inference or representation change;
- closure defect -> proof obstruction or cost;
- grading descent -> search-priority or lemma refinement;
- retained residue -> structured failure telemetry;
- class enlargement -> certified addition of representational capacity.

The useful commonality is procedural: failure is mapped, graded, and reused.

21. Falsifiability and Counterexample Discipline

The demonstrator must make itself easy to falsify.

A critic should be able to test:

- whether the kernel accepts an invalid proof;
- whether a claimed derived rule lacks a primitive expansion;
- whether a minimum was asserted without exhaustive bounded enumeration;
- whether the cost metric changed between classes;
- whether duplicate elimination removed non-equivalent proofs;
- whether a claimed strict decrease disappears under primitive normalization;
- whether the self-sharpening layer can bypass the kernel;
- whether an unresolved search was mislabeled as positive residue;
- whether an axiom extension was mislabeled as conservative representation enlargement.

Every experimental claim should therefore export:

- source code version;
- theory and rule-set identifiers;
- target formula encoding;
- enumeration order;

- search bound;
- accepted proof noun;
- primitive expansion;
- kernel trace;
- cost vector;
- minimality certificate for the exhausted domain;
- registry and telemetry hash.

The system earns strength by exposing all of these surfaces, not by presenting itself as infallible.

22. Implementation Plan

Phase 1 - Minimal kernel and enumerator.

Implement propositional formulas, a Hilbert basis, modus ponens, canonical serialization, exhaustive bounded enumeration, and proof verification.

Phase 2 - Residue and telemetry layer.

Add cost vectors, stage-indexed best values, rejection taxonomy, duplicate normalization, proof-family storage, and certificate export.

Phase 3 - Self-sharpening layer.

Detect repeated subproofs, propose macros, verify primitive expansions, rerun the search, and compare indexed minima.

Phase 4 - Three.js visual demonstrator.

Render the proof hypergraph, search frontier, rejected branches, current minimum, recurring motifs, and class-enlargement shortcuts.

Phase 5 - First-order logic and ZFC kernel.

Add de Bruijn variables, quantifiers, substitution hygiene, equality, ZFC axiom schemas, and finite proof checking.

Phase 6 - Noun and Nock compatibility.

Replace or mirror JavaScript data structures with canonical nouns, implement a minimal Nock evaluator or bridge, and verify cross-implementation agreement.

The first publishable demonstration requires only Phases 1 through 4. ZFC and Nock can appear as formally specified extensions with partial implementation status if necessary, provided the paper distinguishes implemented results from planned architecture.

23. Contribution Statement

Version 2 integrates two calibrated residue instruments. The proposed contribution is not a new logic and not a claim to decide all mathematics. It is the integration of several individually familiar mechanisms into a single formally indexed architecture:

30. an infinite computable registry of finite proof objects and search histories;
31. an immutable proof-verification kernel;

32. a residue calculus indexed by theory, representation class, cost, target, and search stage;
33. failure telemetry retained as structured process data;
34. graph-theoretic shortest-route analysis over proof and lemma relations;
35. conservative self-sharpening through certified lemma and macro discovery;
36. noun-level granularization suitable for Nock;
37. a minimal browser demonstrator that visibly separates theoremhood, proof cost, and bounded search state.

The paper's broad direction and the demonstrator's minimal direction are intentionally complementary. The theory describes the expandable proof registry. The demonstrator proves only the smallest utility claim required to justify the architecture.

24. Conclusion

A proof engine need not treat unsuccessful search as discarded noise. It can preserve the structure of failure, redundancy, and cost, index that structure to a declared representation class, and use it to sharpen later search without relaxing verification.

The resulting system is universal in the constructive sense that it can enumerate and verify every finite proof object in its chosen recursively presented calculus. It remains disciplined about what no finite stage can establish. Its self-sharpening behavior occurs through conservative representation change: recurring proof paths become lemmas, lemmas become certified shortcuts, and shortest represented routes descend while the primitive kernel remains fixed.

The paired Version 2 demonstrator can establish this architecture's central utility with one theorem, two proof classes, one obstruction functional, and one certified strict descent. A JavaScript and Three.js implementation can make the registry visible as a structured proof space rather than a hidden search procedure. ZFC and Nock then provide the broader horizon: a computably presented infinite registry of finite formal proofs, granularized into a minimal universal substrate, with every improvement recorded as a reproducible change in indexed residue.

The appropriate claim is therefore neither philosophical bookkeeping nor universal mathematical conquest. It is a precise executable method for turning proof-search failure into graded information and graded information into certified representational improvement.

Appendix A. Minimal Data Model

Formula record:

- formula_id
- canonical_encoding
- free_variables
- symbol_count

Proof record:

- proof_id
- theory_version
- representation_class

- target_formula_id
- rule_id
- premise_proof_ids
- primitive_expansion_id
- kernel_status
- rejection_code
- cost_vector
- enumeration_stage
- generator_strategy
- equivalence_class
- content_hash

Experiment record:

- experiment_id
- source_version
- target_formula_id
- theory_version
- class_R0
- class_R1
- cost_metric
- search_bound
- best_R0
- best_R1
- strict_drop
- certificate_bundle_hash

Appendix B. Minimal Pseudocode

```
function verify(proof):
  if proof is axiom-instance:
    return schema_match(proof.formula)
  verified_premises = [verify(p) for p in proof.premises]
  if any premise fails:
    return reject(F5)
  return check_primitive_rule(proof.rule, proof.premises, proof.formula)

function enumerate(target, class_R, bound_B):
  best = infinity
  witness = none
  telemetry = empty_registry()
  for candidate in canonical_candidates(class_R, bound_B):
    verdict = verify(expand(candidate))
    telemetry.record(candidate, verdict)
    if verdict is accept and conclusion(candidate) == target:
      cost = obstruction(candidate)
      if cost < best:
```

```
            best = cost
            witness = candidate
    return [best, witness, telemetry, bounded_minimality_certificate()]

function sharpen(telemetry, class_R):
    motifs = frequent_valid_subproofs(telemetry)
    proposals = rank_by_expected_route_reduction(motifs)
    certified = []
    for proposal in proposals:
        expansion = derive_primitive_expansion(proposal)
        if verify(expansion) == accept:
            certified.append(proposal)
    return class_R union certified
```

References to Prior Program Documents

Semita, Lu. The Residue Thesis: Irreducibility as the Invariant of Re-Description. EmergenceByDesign, working draft R1, July 2026.

Semita, Lu, and Jenny Lorraine Nielsen. Operator-Agnostic Geometry, Topological Obstruction, and Why Completeness Is Not Comprehension. Registry Studies, Lambda / NSAF / TUFT, working draft, June 2026.

Semita, Lu. Recursive Registry Completion: Inverse Limits, Dynamical Witnessing, and Irreducible Residue in Lambda / NSAF / TUFT. EmergenceByDesign, laboratory notes record, July 2026.

Semita, Lu. Structural Registry of Nested Geometries: Polygonal, Polyhedral, and Harmonic Registries as Scale-Invariant Incidence Structures. Working Draft, Theorem-Level Version 2, June 2026.

Semita, Lu. Positional Recoverability in Binary-Addressed Registries: A Symmetry Obstruction, with a Tight Worked Instance over the Finite-Type Cartan Matrices. EmergenceByDesign, Version 1.0, July 2026.

Semita, Lu. Nested Geometry Representation of the Exceptional Lie Group E8. Supplement to Structural Registry of Nested Geometries, June 2026.